COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010: Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	7/16/26	**DATE SUBMITTED:**	7/20/26
NAME:	Santos, Paterno Miguel R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 2-1. Initial Driver Program

Screenshot

CODE:

```
#include <iostream>
#include <string.h>

class Student{
    private:
    std::string studentName;
    int studentAge;
    public:
    //constructor
    Student(std::string newName ="John Doe", int newAge=18){
    studentName = std::move(newName);
    studentAge = newAge;
    std::cout << "Constructor Called." << std::endl;
};
//deconstructor
~Student(){
    std::cout << "Destructor Called." << std::endl;
}
//Copy Constructor
Student(const Student &copyStudent){
    std::cout << "Copy Constructor Called" << std::endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
}
//Display Attributes
void printDetails(){
    std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};
int main() {
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;
    return 0;
}
```

OUTPUT:

```
PS C:\msys64\ucrt64\bin> C:\Use
rs\TIPQC\SANTOS.cpp\output\HOA_
2.1.1.exe
Constructor Called.
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\msys64\ucrt64\bin>
```

Observation	This code is an example of a constructor, deconstructor and copy constructor in a class. The constructor is used to create an object inside the Student class and when it is declared in the int main, it calls insert variables inside. The copy constructor then copies the data stored inside the constructor and is called to copy the age for student 2. Then the destructor destroys all objects created.

Table 2-2. Modified Driver Program with Student Lists

Screenshot	**CODE:**
	`#include <iostream>`
	`#include <string.h>`
	
	`class Student{`
	`private:`
	`std::string studentName;`
	`int studentAge;`
	`public:`
	`//constructor`
	`Student(std::string newName ="John Doe", int newAge=18){`
	`studentName = std::move(newName);`
	`studentAge = newAge;`
	`std::cout << "Constructor Called." << std::endl;`
	`};`
	`//deconstructor`
	`~Student(){`
	`std::cout << "Destructor Called." << std::endl;`
	`}`
	`//Copy Constructor`
	`Student(const Student &copyStudent){`
	`std::cout << "Copy Constructor Called" << std::endl;`
	`studentName = copyStudent.studentName;`
	`studentAge = copyStudent.studentAge;`
	`}`
	`//Display Attributes`
	`void printDetails(){`
	`std::cout << this->studentName << " " << this->studentAge << std::endl;`
	`}`
	`};`
	`int main() {`
	`const size_t j = 5;`
	`Student studentList[j] = {};`
	`std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};`
	`int ageList[j] = {15, 16, 18, 19, 16};`
	`15`
	`return 0;`
	`}`

	OUTPUT:
	`PS C:\msys64\ucrt64\bin> C:\Users\TIPQC\SANTOS.cpp\output\HOA_2.1.2.exe`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Destructor Called.`
	`Destructor Called.`
	`Destructor Called.`
	`Destructor Called.`
	`Destructor Called.`

Observation	The constructor here is used to input values inside an array into a fixed setting for the student name list and the student age list. It goes through every index of both student name list and student age list in an alternate path. It then deletes everything besides the student age list using the deconstructor.

Table 2-3. Final Driver Program

Loop A	**CODE:**
	`for(int i = 0; i < j; i++){ //loop A`
	`Student *ptr = new Student(namesList[i], ageList[i]);`
	`studentList[i] = *ptr;`
	`}`

Observation	Since the code above turns the stacked memory for all five indexes of both lists as “John Doe” for student name and “18” for student age, this loop allows us to put the declared names list and age list in the int main by inserting both values in every iteration. Using a pointer, the code points to the data that is inserted inside the heap memory. Then dereferencing the data to input it inside an index in every iteration.

Loop B	**CODE:**
	`for(int i = 0; i < j; i++){ //loop B`
	`studentList[i].printDetails();`
	`}`
	`return 0;`
	`}`

Observation	This loop prints the details of every iteration, starting from 0. It prints the student name right beside the student age inside the current index.
Output	**CODE:** PS C:\msys64\ucrt64\bin> C:\Users\TIPQC\SANTOS.cpp\output\HOA_2.1.3.exe Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Carly 15 Freddy 16 Sam 18 Zack 19 Cody 16 Destructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called.
Observation	The first five “Constructor Called.” output is caused by inserting “John Doe” and “18” in the stacked array while the remaining five is caused by inserting the new values to the array that came from the heap. The next printed output is caused by the “studentList[i].printDetails();” that was looped five times to access every available index inside the array and printed its details. The destructor then destroys the contents of the constructor but does not destroy the data inside the heap.

Table 2-4. Modifications/Corrections Necessary

Modifications	**CODE:**

```cpp
#include <iostream>
#include <string>

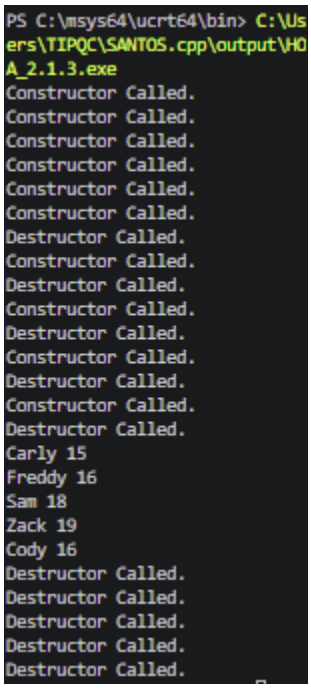
class Student{
    private:
    std::string studentName;
    int studentAge;
    public:
    //constructor
    Student(std::string newName ="John Doe", int newAge=18){
    studentName = std::move(newName);
    studentAge = newAge;
    std::cout << "Constructor Called." << std::endl;
};
//deconstructor
~Student(){
    std::cout << "Destructor Called." << std::endl;
}
//Copy Constructor
Student(const Student &copyStudent){
    std::cout << "Copy Constructor Called" << std::endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
}
//Display Attributes
void printDetails(){
    std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};
int main() {
const size_t j = 5;
Student studentList[j] = {};
std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
int ageList[j] = {15, 16, 18, 19, 16};

for(int i = 0; i < j; i++){ //loop A
Student *ptr = new Student(namesList[i], ageList[i]);
studentList[i] = *ptr;

delete ptr;
ptr = nullptr;
}

for(int i = 0; i < j; i++){ //loop B
studentList[i].printDetails();
}

return 0;
}
```

	OUTPUT: 
Observation	I changed “<string.h>” to “<string> since the code only uses “std::string”. I also added “delete ptr;” inside loop A to destroy heap memory after the transfer then coded “ptr = nullptr;” to avoid dangling pointers afterwards.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

Fruit Class

```
// Problem 1.1: Fruit class
class Fruit {
public:
    std::string name; int price; int amount;

    // Constructor
    Fruit(std::string n, int p, int a) {name = n; price = p; amount = a;}

    // Copy Constructor
    Fruit(const Fruit& x) {name = x.name; price = x.price; amount = x.amount;}

    // Copy Assignment
    void operator=(const Fruit& x) {name = x.name; price = x.price; amount = x.amount;}

    // Destructor
    ~Fruit(){std::cout << name << " is deleted\n";}

    int sum(){return price * amount;}
};
```

Vegetable Class

```
// Problem 1.2: Vegetable class
class Vegetable {
public:
    std::string name; int price; int amount;

    // Constructor
    Vegetable(std::string n, int p, int a) {name = n; price = p; amount = a;}

    // Copy Constructor
    Vegetable(const Vegetable& x) {name = x.name; price = x.price; amount = x.amount;}

    // Copy Assignment
    void operator=(const Vegetable& x) {name = x.name; price = x.price; amount = x.amount;}

    // Destructor
    ~Vegetable() {std::cout << name << " is deleted\n";}
    int sum() {return price * amount;}
};
```

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

Grocery List Array

```
int main() {
    // Problem 2: GroceryList array
    void* GroceryList[4];
    Fruit* apple = new Fruit("Apple", 10, 7);
    Fruit* banana = new Fruit("Banana", 10, 8);
    Vegetable* broccoli = new Vegetable("Broccoli", 60, 12);
    Vegetable* lettuce = new Vegetable("Lettuce", 50, 10);

    GroceryList[0] = apple;
    GroceryList[1] = banana;
    GroceryList[2] = broccoli;
    GroceryList[3] = lettuce;
    Fruit* fruits[2];
    Vegetable* vegetables[2];
    fruits[0] = apple;
    fruits[1] = banana;
    vegetables[0] = broccoli;
    vegetables[1] = lettuce;
```

Display

```
// Display of Grocery List
std::cout << "Grocery List\n\n" << apple->name << " "
          << apple->price << " x "  << apple->amount << " = "
          << apple->sum() << "\n" << banana->name << " "
          << banana->price << " x "<< banana->amount << " = "
          << banana->sum() << "\n" << broccoli->name << " "
          << broccoli->price << " x " << broccoli->amount << " = "
          << broccoli->sum() << "\n" << lettuce->name << " "
          << lettuce->price << " x " << lettuce->amount << " = "
          << lettuce->sum() << "\n";
```

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

TotalSum Function

```
// Problem 3.1: TotalSum func.
int TotalSum(Fruit* fruits[], Vegetable* vegetables[]) {
    int total = 0;

    total = total + fruits[0]->sum();
    total = total + fruits[1]->sum();
    total = total + vegetables[0]->sum();
    total = total + vegetables[1]->sum();

    return total;
}
```

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned

Deleting Lettuce

```
// Problem 4: Deleting Lettuce
std::cout << "\nDeleting Lettuce\n";
delete lettuce; vegetables[1] = 0;
```

Full Code:

```cpp
#include <iostream>
#include <string>

// Problem 1.1: Fruit class
class Fruit {
public:
    std::string name; int price; int amount;

    // Constructor
    Fruit(std::string n, int p, int a) {name = n; price = p; amount = a;}

    // Copy Constructor
    Fruit(const Fruit& x) {name = x.name; price = x.price; amount = x.amount;}

    // Copy Assignment
    void operator=(const Fruit& x) {name = x.name; price = x.price; amount = x.amount;}

    // Destructor
    ~Fruit(){std::cout << name << " is deleted\n";}

    int sum(){return price * amount;}
};
```

```cpp
// Problem 1.2: Vegetable class
class Vegetable {
public:
    std::string name; int price; int amount;

    // Constructor
    Vegetable(std::string n, int p, int a) {name = n; price = p; amount = a;}

    // Copy Constructor
    Vegetable(const Vegetable& x) {name = x.name; price = x.price; amount = x.amount;}

    // Copy Assignment
    void operator=(const Vegetable& x) {name = x.name; price = x.price; amount = x.amount;}

    // Destructor
    ~Vegetable() {std::cout << name << " is deleted\n";}
    int sum() {return price * amount;}
};

// Problem 3.1: TotalSum func.
int TotalSum(Fruit* fruits[], Vegetable* vegetables[]) {
    int total = 0;

    total = total + fruits[0]->sum();
    total = total + fruits[1]->sum();
    total = total + vegetables[0]->sum();
    total = total + vegetables[1]->sum();

    return total;
```

```
}

int main() {
    // Problem 2: GroceryList array
    void* GroceryList[4];
    Fruit* apple = new Fruit("Apple", 10, 7);
    Fruit* banana = new Fruit("Banana", 10, 8);
    Vegetable* broccoli = new Vegetable("Broccoli", 60, 12);
    Vegetable* lettuce = new Vegetable("Lettuce", 50, 10);

    GroceryList[0] = apple;
    GroceryList[1] = banana;
    GroceryList[2] = broccoli;
    GroceryList[3] = lettuce;
    Fruit* fruits[2];
    Vegetable* vegetables[2];
    fruits[0] = apple;
    fruits[1] = banana;
    vegetables[0] = broccoli;
    vegetables[1] = lettuce;

    // Display of Grocery List
    std::cout << "Grocery List\n\n" << apple->name << " "
              << apple->price << " x "  << apple->amount << " = "
              << apple->sum() << "\n" << banana->name << " "
              << banana->price << " x "<< banana->amount << " = "
              << banana->sum() << "\n" << broccoli->name << " "
              << broccoli->price << " x " << broccoli->amount << " = "
              << broccoli->sum() << "\n" << lettuce->name << " "
```

```
            << lettuce->price << " x " << lettuce->amount << " = "
            << lettuce->sum() << "\n";

    // Problem 3.2: Output
    std::cout << "\nTotal price: " << TotalSum(fruits, vegetables) << "\n";

    // Problem 4: Deleting Lettuce
    std::cout << "\nDeleting Lettuce\n";
    delete lettuce; vegetables[1] = 0;

    std::cout << "Updated total price: " << apple->sum() + banana->sum() + broccoli->sum() << "\n";

    delete apple; fruits[0] = 0;
    delete banana; fruits[1] = 0;
    delete broccoli; vegetables[0] = 0;

    return 0;
}
```

Analysis: It starts by creating two objects with constructors, destructors, copy assignment and copy constructors. The array stores the memory addresses of the objects from the heap to allow the program to access and modify things using the pointers instead of copying the objects. The “TotalSum” function uses the pointers to calculate the total price, creating a stack memory while the function runs. Then the destructor is called to remove the lettuce to remove it from the heap.

Summary: I have learned how constructors and destructors work inside a code. I have learned how to allocate memories but I am still confused how pointers work. I have learned how to use it in a code and what it does, but I can not comprehend what it really does. Being able to visualize it also helps me learn how the data is being transferred around.

Feedback: How well did you think you did in this activity? What are your areas for improvement?
I think I did well in understanding the concept of the lesson but I am still having trouble understanding how it works. I also find it confusing as to why we need to have a copy assignment and a copy constructor since I do not see it getting called in the main function anywhere. I can still improve in understanding pointers, copy assignment and copy constructor since I encountered multiple errors and it took me multiple tries to get it right while also taking a long time analyzing and deleting lines of codes to see what makes it not work.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used it for code visualization to understand what is happening inside the code**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.